ಆಕ್ಟಿವೇಶನ್, ಲಾಸ್ ಮತ್ತು ಆಪ್ಟಿಮೈಜರ್‌ಗಳು

ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಅನ್ನು ಸರಿಯಾಗಿ ತರಬೇತಿ ಮಾಡುವುದು ಹೇಗೆ?

ಸರಳ ಕನ್ನಡದಲ್ಲಿ, ದಿನನಿತ್ಯದ ಉದಾಹರಣೆಗಳು ಮತ್ತು ಹಂತ-ಹಂತದ ಲೆಕ್ಕಗಳೊಂದಿಗೆ

“Lecture 4: Activation and Loss Functions — Batch, Stochastic and Mini-Batch Training Algorithms”

ಉಪನ್ಯಾಸ ಸ್ಲೈಡ್‌ಗಳನ್ನು ಆಧರಿಸಿದ ಕನ್ನಡ ಅಧ್ಯಯನ ಮಾರ್ಗದರ್ಶಿ

ಪರಿವಿಡಿ

೧. ದೊಡ್ಡ ಚಿತ್ರ: ಮೂರು ಪ್ರಮುಖ ಆಯ್ಕೆಗಳು

೨. Activation function ಏಕೆ ಬೇಕು?

೩. ಕ್ಲಾಸಿಕ್ ಆಕ್ಟಿವೇಶನ್‌ಗಳು: Step, Sigmoid, Tanh, ReLU

೪. ಡೀಪ್ ನೆಟ್‌ವರ್ಕ್‌ನಲ್ಲಿ Vanishing Gradient

೫. ಔಟ್‌ಪುಟ್ ಪದರದ ಆಕ್ಟಿವೇಶನ್: ಕಾರ್ಯಕ್ಕೆ ತಕ್ಕಂತೆ (Linear, Sigmoid, Softmax)

೬. Loss function ಎಂದರೇನು?

೭. ಸಂಖ್ಯೆ ಊಹಿಸುವ ತಪ್ಪುಗಳು: MAE, MSE, RMSE, MSLE

೮. ಶೇಕಡಾ ತಪ್ಪುಗಳು: MPE ಮತ್ತು MAPE

೯. Entropy ಮತ್ತು Cross-Entropy (ವರ್ಗೀಕರಣದ loss)

೧೦. Gradient Descent ನ ಸಮಸ್ಯೆಗಳು: Local minimum, Saddle point, Overfitting

೧೧. Batch Gradient Descent

೧೨. Stochastic ಮತ್ತು Mini-Batch Gradient Descent (ಒಂದೇ ಉದಾಹರಣೆಯ ಲೆಕ್ಕದೊಂದಿಗೆ)

೧೩. Momentum ಮತ್ತು Nesterov (NAG)

೧೪. AdaGrad, RMSProp, Adadelta

೧೫. Adam ಮತ್ತು Nadam

೧೬. ಯಾವುದನ್ನು ಆರಿಸಬೇಕು? ಹೋಲಿಕೆ ಮತ್ತು ಮಿತಿಗಳು

೧೭. ಸಾರಾಂಶ, ಪದಕೋಶ ಮತ್ತು ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

೧. ದೊಡ್ಡ ಚಿತ್ರ: ಮೂರು ಪ್ರಮುಖ ಆಯ್ಕೆಗಳು

ಹಿಂದಿನ ಉಪನ್ಯಾಸದಲ್ಲಿ ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಕಲಿಯುವ ಚಕ್ರ (ಊಹಿಸು → ತಪ್ಪು ಅಳೆ → ಹಿಂದಕ್ಕೆ ಹಂಚು → ತೂಕ ಬದಲಿಸು) ನೋಡಿದೆವು. ಆ ಚಕ್ರದಲ್ಲಿ ಮೂರು ಜಾಗಗಳಲ್ಲಿ ನಾವು **ಆಯ್ಕೆ ಮಾಡಬೇಕು**. ಆ ಆಯ್ಕೆಗಳೇ ಈ ಉಪನ್ಯಾಸದ ವಿಷಯ:

ಆಯ್ಕೆ	ಯಾವ ಪ್ರಶ್ನೆಗೆ ಉತ್ತರ?	ಉದಾಹರಣೆಗಳು
Activation function	ನ್ಯೂರಾನ್ ತನ್ನ ಮೊತ್ತವನ್ನು ಹೇಗೆ ಔಟ್‌ಪುಟ್ ಆಗಿ ಬದಲಿಸುತ್ತದೆ?	ReLU, Sigmoid, Tanh, Softmax, Linear
Loss function	ನೆಟ್‌ವರ್ಕ್ ಎಷ್ಟು ತಪ್ಪು ಎಂದು ಹೇಗೆ ಅಳೆಯುವುದು?	MAE, MSE, Cross-Entropy
Optimizer	ತಪ್ಪು ಕಡಿಮೆ ಮಾಡಲು ತೂಕಗಳನ್ನು ಹೇಗೆ ಬದಲಿಸುವುದು?	SGD, Momentum, Adam

> **ಕಥೆ: ಪರೀಕ್ಷೆಗೆ ತಯಾರಿ**
>
> ಒಬ್ಬ ವಿದ್ಯಾರ್ಥಿ ಪರೀಕ್ಷೆಗೆ ತಯಾರಾಗುತ್ತಿದ್ದಾನೆ. **ಆಕ್ಟಿವೇಶನ್** = ಅವನು ಉತ್ತರವನ್ನು ಯಾವ ರೂಪದಲ್ಲಿ ಬರೆಯುತ್ತಾನೆ (ಹೌದು/ಇಲ್ಲ, ಒಂದು ಸಂಖ್ಯೆ, ಅಥವಾ ಆಯ್ಕೆಗಳ ಪಟ್ಟಿ). **ಲಾಸ್** = ಶಿಕ್ಷಕರು ಅಂಕ ಕಡಿತ ಮಾಡುವ ವಿಧಾನ (ಸಣ್ಣ ತಪ್ಪಿಗೆ ಸ್ವಲ್ಪ, ದೊಡ್ಡ ತಪ್ಪಿಗೆ ಹೆಚ್ಚು). **ಆಪ್ಟಿಮೈಜರ್** = ತಪ್ಪುಗಳಿಂದ ಕಲಿತು ಅಭ್ಯಾಸವನ್ನು ಸುಧಾರಿಸುವ ವಿಧಾನ (ಒಂದೊಂದು ಪ್ರಶ್ನೆ ನೋಡಿ ತಿದ್ದಿಕೊಳ್ಳುವುದೋ, ಇಡೀ ಪೇಪರ್ ನೋಡಿ ತಿದ್ದಿಕೊಳ್ಳುವುದೋ?). ಮೂರೂ ಸರಿಯಾಗಿದ್ದರೆ ಮಾತ್ರ ಒಳ್ಳೆಯ ಫಲಿತಾಂಶ.

೨. Activation function ಏಕೆ ಬೇಕು?

ಆಕ್ಟಿವೇಶನ್ ಫಂಕ್ಷನ್ ಎರಡು ಕೆಲಸ ಮಾಡುತ್ತದೆ:

- **ನಾನ್-ಲೀನಿಯರಿಟಿ (non-linearity) ಸೇರಿಸುತ್ತದೆ** — ಇದರಿಂದ ನೆಟ್‌ವರ್ಕ್ ಕೇವಲ ನೇರ ರೇಖೆಗಳಲ್ಲ, ಸಂಕೀರ್ಣ ಮಾದರಿಗಳನ್ನೂ ಕಲಿಯಬಲ್ಲದು.
- **ನ್ಯೂರಾನ್ ಸಕ್ರಿಯವಾಗಬೇಕೇ** (ಸಂಕೇತವನ್ನು ಮುಂದಕ್ಕೆ ಕಳುಹಿಸಬೇಕೇ) ಎಂದು ನಿರ್ಧರಿಸುತ್ತದೆ.

ಮೂರು ಸಾಮಾನ್ಯ ಪ್ರಶ್ನೆಗಳು

ಪ್ರಶ್ನೆ	ಉತ್ತರ
೧. ಪ್ರತಿ ನ್ಯೂರಾನ್‌ಗೂ ಬೇರೆ ಬೇರೆ activation ಇರಬಹುದೇ?	ಸಾಧ್ಯ, ಆದರೆ ಸಾಮಾನ್ಯವಾಗಿ **ಮರೆ ಪದರಗಳ ಎಲ್ಲ ನ್ಯೂರಾನ್‌ಗಳಿಗೆ ಒಂದೇ** ಫಂಕ್ಷನ್ ಬಳಸುತ್ತಾರೆ.
೨. ಲೀನಿಯರ್ ಫಂಕ್ಷನ್‌ಗಳನ್ನು ಮಾತ್ರ ಏಕೆ ಬಳಸಬಾರದು?	ಎಷ್ಟೇ ಪದರಗಳಿದ್ದರೂ ಅದು **ಒಂದೇ ಲೀನಿಯರ್ ಮಾದರಿ**ಯಂತೆ ವರ್ತಿಸುತ್ತದೆ (ಕೆಳಗೆ ಲೆಕ್ಕ ನೋಡಿ).
೩. ಔಟ್‌ಪುಟ್ ಪದರಕ್ಕೆ ಯಾವುದು?	ಕಾರ್ಯದ ಪ್ರಕಾರವನ್ನು ಅವಲಂಬಿಸಿದೆ (ವರ್ಗೀಕರಣ ಅಥವಾ ಸಂಖ್ಯೆ ಊಹೆ) — ಭಾಗ ೫ ನೋಡಿ.

ಲೆಕ್ಕ: ಲೀನಿಯರ್ ಪದರಗಳನ್ನು ಜೋಡಿಸಿದರೆ ಏನಾಗುತ್ತದೆ?

ಎರಡು ಪದರಗಳು, ಪ್ರತಿ ಪದರದಲ್ಲಿ ಒಂದೇ ನ್ಯೂರಾನ್, ಆಕ್ಟಿವೇಶನ್ ಇಲ್ಲ (y = ತೂಕ × ಇನ್‌ಪುಟ್). ಪದರ 1 ರ ತೂಕ 2, ಪದರ 2 ರ ತೂಕ 3.

```
ಇನ್‌ಪುಟ್ x = 5
ಪದರ 1 : y1 = 2 × 5  = 10
ಪದರ 2 : y2 = 3 × 10 = 30

ಒಂದೇ ಪದರ w = 6 ಇದ್ದರೂ ಅದೇ ಉತ್ತರ:  6 × 5 = 30
```

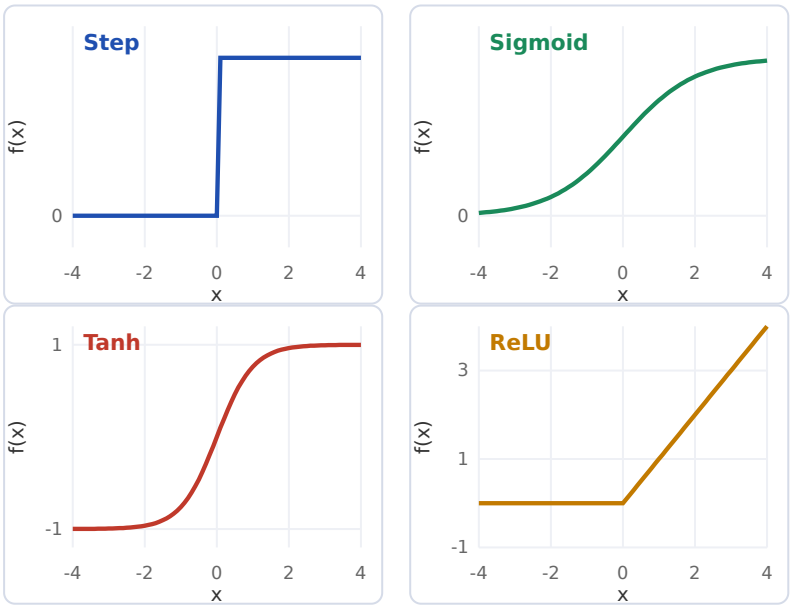
ಅಂದರೆ ಎರಡು ಪದರಗಳು ಒಂದೇ ಪದರದಂತೆ — ಪದರ ಸೇರಿಸಿದ್ದು ವ್ಯರ್ಥ. ಆದರೆ ಮಧ್ಯದಲ್ಲಿ ReLU ಇಟ್ಟರೆ (ಋಣ ಸಂಖ್ಯೆಗಳನ್ನು 0 ಮಾಡುವ) ಇನ್‌ಪುಟ್ x = −5 ಗೆ: ಪದರ 1 → −10 → ReLU → 0 → ಪದರ 2 → 0. ಆದರೆ ಒಂದೇ ಲೀನಿಯರ್ ಪದರ −30 ಕೊಡುತ್ತಿತ್ತು. ಈಗ ನೆಟ್‌ವರ್ಕ್ ಒಂದು ನೇರ ರೇಖೆಗಿಂತ ಹೆಚ್ಚು ಸಂಕೀರ್ಣವಾಗಿ ವರ್ತಿಸುತ್ತಿದೆ.

ಹೋಲಿಕೆ

ನೇರ ರಸ್ತೆಗಳನ್ನು ಎಷ್ಟೇ ಜೋಡಿಸಿದರೂ ಒಂದು ನೇರ ರಸ್ತೆಯೇ ಆಗುತ್ತದೆ. ಘಾಟಿ ರಸ್ತೆಯ ತಿರುವುಗಳು ಬೇಕಾದರೆ ರಸ್ತೆಯ ನಡುವೆ "ತಿರುವು" ಬೇಕು. ಆಕ್ಟಿವೇಶನ್ ಫಂಕ್ಷನ್ ಆ ತಿರುವು.

ಸಾಮಾನ್ಯ ಆಯ್ಕೆಗಳು: ಮರೆ ಪದರಗಳಿಗೆ `selu, tanh, elu, relu` ; ಔಟ್‌ಪುಟ್ ಪದರಕ್ಕೆ `softmax, linear, sigmoid` .

೩. ಕ್ಲಾಸಿಕ್ ಆಕ್ಟಿವೇಶನ್‌ಗಳು: Step, Sigmoid, Tanh, ReLU



ಚಿತ್ರ ೧: ನಾಲ್ಕು ಪ್ರಮುಖ ಆಕ್ಟಿವೇಶನ್ ಫಂಕ್ಷನ್‌ಗಳ ಆಕಾರ

ಬೇರೆ ಬೇರೆ ಇನ್‌ಪುಟ್‌ಗೆ ಬೆಲೆಗಳು (ಲೆಕ್ಕ ಹಾಕಿದ್ದು)

x	Step	Sigmoid	Tanh	ReLU	Leaky ReLU (0.1)
−2	0	0.119	−0.964	0	−0.2
−1	0	0.269	−0.762	0	−0.1
0	0	0.500	0.000	0	0
1	1	0.731	0.762	1	1
2	1	0.881	0.964	2	2

ಒಂದೊಂದರ ಪರಿಚಯ

ಫಂಕ್ಷನ್	ಸೂತ್ರ ಮತ್ತು ವ್ಯಾಪ್ತಿ	ಒಳ್ಳೆಯದು	ಸಮಸ್ಯೆ
Step (Heaviside)	$x > 0$ ಆದರೆ 1, ಇಲ್ಲದಿದ್ದರೆ 0	ಅತಿ ಸರಳ — ಸ್ವಿಚ್‌ನಂತೆ	ವ್ಯುತ್ಪನ್ನ ಬಹುತೇಕ ಎಲ್ಲೆಡೆ 0 → ಕಲಿಕೆ ಅಸಾಧ್ಯ
Sigmoid	$1/(1+e^{-x})$, ವ್ಯಾಪ್ತಿ 0 ರಿಂದ 1	ಸಂಭವನೀಯತೆಯಂತೆ ಓದಬಹುದು	ಗ್ರೇಡಿಯಂಟ್ ಗರಿಷ್ಠ 0.25 → vanishing gradient
Tanh	ವ್ಯಾಪ್ತಿ −1 ರಿಂದ 1, 0 ಸುತ್ತ ಸಮ್ಮಿತಿ	Sigmoid ಗಿಂತ ವೇಗದ ಕಲಿಕೆ; ಧನ-ಋಣ ಎರಡೂ ಉಳಿಸುತ್ತದೆ (RNN, LSTM, GRU ನಲ್ಲಿ ಮುಖ್ಯ)	ಇದರಲ್ಲೂ vanishing gradient; ಸರಿಯಾದ ತೂಕ ಆರಂಭ/ಇನ್‌ಪುಟ್ normalization ಇಲ್ಲದಿದ್ದರೆ exploding
ReLU	$\max(0, x)$	ಸರಳ, ವೇಗ; $x > 0$ ಗೆ ವ್ಯುತ್ಪನ್ನ = 1 → vanishing ಇಲ್ಲ; ಡೀಪ್ ನೆಟ್‌ವರ್ಕ್‌ಗೆ ಸೂಕ್ತ	**Dead neurons** (ಯಾವಾಗಲೂ 0 ಕೊಡುವ ನ್ಯೂರಾನ್‌ಗಳು)

ಹೋಲಿಕೆ: ReLU ಮತ್ತು ಸತ್ತ ನ್ಯೂರಾನ್

ReLU ಒಂದು ಏಕಮುಖ ಕವಾಟದಂತೆ: ಧನ ಸಂಕೇತ ಹಾದುಹೋಗುತ್ತದೆ, ಋಣ ಸಂಕೇತ ನಿಲ್ಲುತ್ತದೆ. ಒಂದು ನ್ಯೂರಾನ್‌ಗೆ ಯಾವಾಗಲೂ ಋಣ ಮೊತ್ತವೇ ಬಂದರೆ ಅದರ ಔಟ್‌ಪುಟ್ ಸದಾ 0, ಗ್ರೇಡಿಯಂಟ್ ಸಹ 0 — ಅದು ಎಂದಿಗೂ ಕಲಿಯುವುದಿಲ್ಲ. ಬೀದಿ ದೀಪದ ಬಲ್ಬ್ ಒಮ್ಮೆ ಸುಟ್ಟು ಹೋದ ಮೇಲೆ ಎಷ್ಟು ವೋಲ್ಟೇಜ್ ಕೊಟ್ಟರೂ ಬೆಳಗದಂತೆ. ತರಬೇತಿಯಲ್ಲಿ ಸುಮಾರು **40% ವರೆಗೆ** ನ್ಯೂರಾನ್‌ಗಳು ಹೀಗೆ ಸಾಯಬಹುದು (ಹೆಚ್ಚಿನ learning rate ಅಥವಾ ಕೆಟ್ಟ ಆರಂಭಿಕ ತೂಕಗಳಿಂದ).

ಪರಿಹಾರ: learning rate ಕಡಿಮೆ ಮಾಡಿ, ಅಥವಾ **Leaky ReLU** (ಋಣ ಭಾಗದಲ್ಲಿ ಸಣ್ಣ ಇಳಿಜಾರು, ಉದಾ: $0.1 \cdot x$), ಅಥವಾ **ELU** ಬಳಸಿ.

ಹೊಸ ದಿಕ್ಕುಗಳು

- **Trainable activations** (Swish, PReLU): ಫಂಕ್ಷನ್‌ನ ಆಕಾರವೂ ತೂಕದಂತೆ ತರಬೇತಿಯಲ್ಲಿ ಕಲಿಯಲ್ಪಡುತ್ತದೆ.
- **Ensemble:** ReLU, sigmoid, tanh ಇವುಗಳ ಮಿಶ್ರಣವನ್ನು ಮಾದರಿಯೇ ಕಲಿಯುತ್ತದೆ.
- **ಸಣ್ಣ ಉಪ-ನೆಟ್‌ವರ್ಕ್‌ಗಳು** (ಉದಾ: Neural Activation Functions) ಆಕ್ಟಿವೇಶನ್ ವಕ್ರವನ್ನೇ ಕಲಿಯುತ್ತವೆ.
- **ಸವಾಲುಗಳು:** ಹೆಚ್ಚು ಲೆಕ್ಕಾಚಾರ, overfitting ಅಪಾಯ; ಆದ್ದರಿಂದ ಮುಖ್ಯ ಪದರಗಳಲ್ಲಿ (ಮಧ್ಯ ಅಥವಾ ಔಟ್‌ಪುಟ್) ಮಾತ್ರ ಬಳಸುತ್ತಾರೆ.

೪. ಡೀಪ್ ನೆಟ್‌ವರ್ಕ್‌ನಲ್ಲಿ Vanishing Gradient

ಹಿಂದಕ್ಕೆ ಹೋಗುವಾಗ (backpropagation) ಪ್ರತಿ ಪದರದಲ್ಲಿ ಆಕ್ಟಿವೇಶನ್‌ನ ವ್ಯುತ್ಪನ್ನವನ್ನು ಗುಣಿಸುತ್ತಾ ಹೋಗುತ್ತೇವೆ. Sigmoid ನ ವ್ಯುತ್ಪನ್ನ $\sigma(x)(1-\sigma(x))$ ಗರಿಷ್ಠ ಮೌಲ್ಯ $x = 0$ ರಲ್ಲಿ: $0.5 \times (1 - 0.5) = \mathbf{¼ = 0.25}$. ಅಂದರೆ ಪ್ರತಿ ಪದರದಲ್ಲಿ ಗ್ರೇಡಿಯಂಟ್ ಕನಿಷ್ಠ 4 ಪಟ್ಟು ಕುಗ್ಗುತ್ತದೆ:

ಪದರಗಳ ಸಂಖ್ಯೆ	ಗರಿಷ್ಠ ಗ್ರೇಡಿಯಂಟ್ ಗುಣಕ	ಸುಮಾರು
1	0.25	¼
2	$0.25 \times 0.25 = 1/16$	0.0625
4	1/256	0.0039
8	0.25^8	0.000015
10	0.25^{10}	0.00000095 (ಬಹುತೇಕ ಸೊನ್ನೆ)

> **ಕಥೆ: ಕಿವಿಯಿಂದ ಕಿವಿಗೆ ಸಂದೇಶ**
> ಹತ್ತು ಜನರ ಸಾಲಿನಲ್ಲಿ ಕೊನೆಯವನು ಮೊದಲನೆಯವನಿಗೆ ಸಂದೇಶ ಕಳುಹಿಸುತ್ತಾನೆ, ಆದರೆ ಪ್ರತಿಯೊಬ್ಬನೂ ಕೇವಲ ಕಾಲು ಭಾಗ ಮಾತ್ರ ಮುಂದಕ್ಕೆ ಹೇಳುತ್ತಾನೆ. ಮೊದಲನೆಯವನಿಗೆ ತಲುಪುವಷ್ಟರಲ್ಲಿ ಏನೂ ಉಳಿದಿರುವುದಿಲ್ಲ! ಹಾಗೆಯೇ ಆರಂಭದ ಪದರಗಳು ತಿದ್ದುಪಡಿ ಸಂದೇಶ ಪಡೆಯುವುದಿಲ್ಲ.

> **ಪರಿಣಾಮ**
> ಆರಂಭದ ಪದರಗಳ ತೂಕಗಳು ಬದಲಾಗುವುದಿಲ್ಲ; ತರಬೇತಿ ಕೇವಲ **ಕೊನೆಯ ಪದರಗಳಲ್ಲಿ** ನಡೆಯುತ್ತದೆ. ಅದಕ್ಕೇ ಡೀಪ್ ನೆಟ್‌ವರ್ಕ್‌ಗಳಲ್ಲಿ ReLU ($x > 0$ ಗೆ ವ್ಯುತ್ಪನ್ನ 1 — ಗುಣಿಸಿದರೂ ಕುಗ್ಗುವುದಿಲ್ಲ) ಜನಪ್ರಿಯ.

೫. ಔಟ್‌ಪುಟ್ ಪದರದ ಆಕ್ಟಿವೇಶನ್: ಕಾರ್ಯಕ್ಕೆ ತಕ್ಕಂತೆ

ನಾವು ಏನನ್ನು ಊಹಿಸಬೇಕು ಎಂಬುದರ ಮೇಲೆ ಔಟ್‌ಪುಟ್ ಪದರದ ಆಕ್ಟಿವೇಶನ್ ನಿರ್ಧಾರವಾಗುತ್ತದೆ. ಎರಡು ಮೂಲ ಕಾರ್ಯಗಳು:

- **Regression (ಸಂಖ್ಯೆ ಊಹೆ):** ಸಂಖ್ಯಾತ್ಮಕ ಮೌಲ್ಯ — ತಾಪಮಾನ, ಬೆಲೆ, ತೂಕ.
- **Classification (ವರ್ಗೀಕರಣ):** ವರ್ಗ — ಬೆಕ್ಕು/ನಾಯಿ, ಸ್ಪ್ಯಾಮ್/ಅಲ್ಲ.

ಕಾರ್ಯ	ಆಕ್ಟಿವೇಶನ್	ಔಟ್‌ಪುಟ್ ಉದಾಹರಣೆ	ನಿಜ ಜೀವನದ ಉದಾಹರಣೆ
Regression	Linear	27.5, 89.0	ನಾಳೆ ಬೆಂಗಳೂರಿನ ತಾಪಮಾನ ಎಷ್ಟು?
Binary classification	Sigmoid	0.92 (ಸಂಭವನೀಯತೆ)	ಈ ಇಮೇಲ್ ಸ್ಪ್ಯಾಮ್ ಹೌದೇ?
Multi-class classification	Softmax	[0.1, 0.8, 0.1]	ಚಿತ್ರದಲ್ಲಿರುವುದು ನಾಯಿಯೋ, ಮೊಲವೋ, ಬೆಕ್ಕೋ? (ಒಂದೇ ಉತ್ತರ)
Multi-label classification	Sigmoid (ಪ್ರತಿ ಲೇಬಲ್‌ಗೆ)	[1, 0, 1]	ಸಿನಿಮಾ: ಆಕ್ಷನ್ ಹೌದು, ಕಾಮಿಡಿ ಇಲ್ಲ, ರೊಮ್ಯಾನ್ಸ್ ಹೌದು (ಹಲವು ಟ್ಯಾಗ್)

(ಅ) Linear — ಸಂಖ್ಯೆ ಊಹಿಸಲು

Linear ಆಕ್ಟಿವೇಶನ್ y = x ಎಂದರೆ ಮೊತ್ತವನ್ನು **ಹಾಗೆಯೇ** ಹೊರಗೆ ಬಿಡುವುದು (x = ಇನ್‌ಪುಟ್·ತೂಕ + ಬಯಾಸ್).

> **ಉದಾಹರಣೆ: ಮನೆ ಬೆಲೆ ಊಹೆ**
>
> ಮನೆ ವಿಸ್ತೀರ್ಣ 1200 ಚ.ಅ., ತೂಕ 0.05 (ಪ್ರತಿ ಚ.ಅ.ಗೆ ₹0.05 ಲಕ್ಷ), ಬಯಾಸ್ 5:
>
> ```
> x = 1200 × 0.05 + 5 = 65     →  Linear: y = 65   (ಅಂದರೆ ₹65 ಲಕ್ಷ)
> ```
>
> Sigmoid ಹಾಕಿದ್ದರೆ ಉತ್ತರ 1 ಕ್ಕಿಂತ ಹೆಚ್ಚಾಗಲು ಸಾಧ್ಯವೇ ಇರುತ್ತಿರಲಿಲ್ಲ — ಬೆಲೆಗೆ ಅದು ತಪ್ಪು. ಹಾಗಾಗಿ ಸಂಖ್ಯೆ ಊಹೆಗೆ Linear.

(ಆ) Softmax — ಹಲವು ವರ್ಗಗಳಲ್ಲಿ ಒಂದನ್ನು ಆರಿಸಲು

Softmax ನ್ಯೂರಾನ್‌ಗಳ ಕಚ್ಚಾ ಅಂಕಗಳನ್ನು (**logits**, ಋಣವೂ ಆಗಬಹುದು) ಸಂಭವನೀಯತೆ ಹಂಚಿಕೆಯಾಗಿ ಬದಲಿಸುತ್ತದೆ; ಎಲ್ಲವುಗಳ ಮೊತ್ತ **1**.

```
y_i = e^(s_i) / ( e^(s_1) + e^(s_2) + … + e^(s_n) )
```

ಉದಾಹರಣೆ: ಪ್ರಾಣಿ ಗುರುತಿಸುವಿಕೆ

ಮೂರು ಔಟ್‌ಪುಟ್ ನ್ಯೂರಾನ್‌ಗಳು: ನಾಯಿ s1 = 2.5, ಮೊಲ s2 = 1.2, ಬೆಕ್ಕು s3 = 0.8

```
ಹಂತ ೧: e^2.5 = 12.182,  e^1.2 = 3.320,  e^0.8 = 2.226
ಹಂತ ೨: ಮೊತ್ತ = 12.182 + 3.320 + 2.226 = 17.728
ಹಂತ ೩: ನಾಯಿ  = 12.182 / 17.728 = 0.687   (ಸುಮಾರು 69%)
          ಮೊಲ   =  3.320 / 17.728 = 0.187   (ಸುಮಾರು 19%)
          ಬೆಕ್ಕು  =  2.226 / 17.728 = 0.126   (ಸುಮಾರು 13%)
ಮೊತ್ತ = 0.687 + 0.187 + 0.126 = 1.000 ✓
```

ನೆಟ್‌ವರ್ಕ್ ಪ್ರಕಾರ ಇದು **ನಾಯಿ** (69%).

ಸ್ಲೈಡ್‌ನಲ್ಲಿನ ಒಂದು ಸಣ್ಣ ವ್ಯತ್ಯಾಸ

ಸ್ಲೈಡ್‌ನಲ್ಲಿ ಇದೇ logits (2.5, 1.2, 0.8) ಗೆ ಫಲಿತಾಂಶ "≈ 0.70, 0.25, 0.05" ಎಂದಿದೆ. ನಿಖರವಾಗಿ ಲೆಕ್ಕ ಹಾಕಿದರೆ ಸರಿಯಾದ ಉತ್ತರ **0.687, 0.187, 0.126** (ಸ್ಲೈಡ್‌ನ 0.25 + 0.05 ಸೇರಿಸಿದರೂ ಮೊತ್ತ 1 ಆಗುವುದಿಲ್ಲ). ಈ ಡಾಕ್ಯುಮೆಂಟ್‌ನಲ್ಲಿ ನಿಖರ ಮೌಲ್ಯಗಳನ್ನೇ ಬಳಸಿದ್ದೇವೆ.

(ಇ) Sigmoid per label — ಹಲವು ಟ್ಯಾಗ್‌ಗಳಿಗೆ (multi-label)

Softmax ನಲ್ಲಿ ವರ್ಗಗಳು ಪರಸ್ಪರ ಸ್ಪರ್ಧಿಸುತ್ತವೆ (ಒಂದು ಏರಿದರೆ ಇನ್ನೊಂದು ಇಳಿಯುತ್ತದೆ). ಒಂದು ಸಿನಿಮಾ ಏಕಕಾಲದಲ್ಲಿ ಆಕ್ಷನ್ ಮತ್ತು ರೊಮ್ಯಾನ್ಸ್ ಎರಡೂ ಆಗಬಹುದು, ಆದ್ದರಿಂದ ಪ್ರತಿ ಟ್ಯಾಗ್‌ಗೆ **ಸ್ವತಂತ್ರ sigmoid**:

```
logits = [ಆಕ್ಷನ್ 2.0, ಕಾಮಿಡಿ −1.0, ರೊಮ್ಯಾನ್ಸ್ 1.5]
sigmoid = [0.881, 0.269, 0.818]   → 0.5 ಮಿತಿಯಿಂದ → [1, 0, 1]   (ಆಕ್ಷನ್ ✓, ಕಾಮಿಡಿ ✗, ರೊಮ್ಯಾನ್ಸ್ ✓)
```

ಇಲ್ಲಿ ಮೊತ್ತ 1 ಆಗಿರಬೇಕಿಲ್ಲ (0.881 + 0.269 + 0.818 = 1.97).

೬. Loss function ಎಂದರೇನು?

Loss (ಅಥವಾ cost) function ಎಂದರೆ ಮಾದರಿಯ ಊಹೆಗಳು ನಿಜವಾದ ಫಲಿತಾಂಶಗಳಿಂದ ಎಷ್ಟು ದೂರ ಇವೆ ಎಂದು ಹೇಳುವ **ಒಂದೇ ಸಂಖ್ಯೆ**. ಸಂಖ್ಯೆ ಚಿಕ್ಕದಾದಷ್ಟೂ ಮಾದರಿ ಒಳ್ಳೆಯದು. ಇದು ಎರಡು ಕೆಲಸ ಮಾಡುತ್ತದೆ:

- **ತರಬೇತಿಗೆ ದಾರಿ ತೋರಿಸುತ್ತದೆ:** Gradient Descent ಈ ಸಂಖ್ಯೆಯನ್ನು ಕಡಿಮೆ ಮಾಡಲು ತೂಕಗಳನ್ನು ಬದಲಿಸುತ್ತದೆ.
- **ಕಾರ್ಯಕ್ಷಮತೆ ಅಳೆಯುತ್ತದೆ:** ಮಾದರಿ ಎಷ್ಟು ಚೆನ್ನಾಗಿದೆ ಎಂದು ತಿಳಿಸುತ್ತದೆ.

ಕಾರ್ಯಕ್ಕೆ ತಕ್ಕ loss

ಕಾರ್ಯ	ಸಾಮಾನ್ಯ loss ಫಂಕ್ಷನ್‌ಗಳು
Regression (ಸಂಖ್ಯೆ)	MAE, MSE, RMSE, MSLE, MPE, MAPE, Huber, Log-Cosh
Binary classification	Binary Cross-Entropy, Hinge, Squared Hinge, Focal Loss
Multi-class classification	Categorical Cross-Entropy (one-hot ಲೇಬಲ್‌ಗೆ), Sparse Categorical Cross-Entropy (ಪೂರ್ಣಾಂಕ ಲೇಬಲ್‌ಗೆ)
Multi-label classification	Binary Cross-Entropy (ಪ್ರತಿ ಲೇಬಲ್‌ಗೆ)
ಸಂಭವನೀಯತೆ ಹಂಚಿಕೆಗಳ ಹೋಲಿಕೆ	Kullback-Leibler (KL) Divergence
ಅಸಮತೋಲನ ವರ್ಗಗಳು (imbalanced)	Focal Loss
ಧ್ವನಿ / ಪಠ್ಯ ಗುರುತಿಸುವಿಕೆ, ಸೀಕ್ವೆನ್ಸ್	CTC Loss, Sequence Loss (RNN/LSTM)
Image segmentation (ಚಿತ್ರದ ಭಾಗ ಗುರುತಿಸುವಿಕೆ)	Dice, Jaccard, Tversky Loss

೭. ಸಂಖ್ಯೆ ಊಹಿಸುವ ತಪ್ಪುಗಳು: MAE, MSE, RMSE, MSLE

ಇಲ್ಲಿ y = ನಿಜವಾದ ಮೌಲ್ಯ, $\hat{y}$ = ನೆಟ್‌ವರ್ಕ್‌ನ ಊಹೆ, n = ಉದಾಹರಣೆಗಳ ಸಂಖ್ಯೆ.

ಹೆಸರು	ಸೂತ್ರ	ಅರ್ಥ
MAE — Mean Absolute Error	$(1/n)\ \Sigma\ \lvert y - \hat{y}\rvert$	ಸರಾಸರಿ ತಪ್ಪಿನ ಗಾತ್ರ (ಅದೇ ಏಕಮಾನದಲ್ಲಿ)
MSE — Mean Squared Error	$(1/n)\ \Sigma\ (y - \hat{y})^2$	ತಪ್ಪುಗಳ ವರ್ಗದ ಸರಾಸರಿ; ದೊಡ್ಡ ತಪ್ಪಿಗೆ ಹೆಚ್ಚು ಶಿಕ್ಷೆ
RMSE — Root MSE	$\sqrt{\text{MSE}}$	MSE ಯನ್ನು ಮತ್ತೆ ಮೂಲ ಏಕಮಾನಕ್ಕೆ ತರುತ್ತದೆ
MSLE — Mean Squared Log Error	$(1/n)\ \Sigma\ (\log(y+1) - \log(\hat{y}+1))^2$	ಅನುಪಾತದ ತಪ್ಪು ಅಳೆಯುತ್ತದೆ; ದೊಡ್ಡ ಮೌಲ್ಯಗಳ ತಪ್ಪಿಗೆ ಮೃದು

ಉದಾಹರಣೆ (ಸ್ಪೈಡ್‌ನ IQ ದತ್ತಾಂಶ)

ಇಬ್ಬರ ನಿಜವಾದ IQ: 90 ಮತ್ತು 105. ನೆಟ್‌ವರ್ಕ್ ಊಹೆ: 95 ಮತ್ತು 100.

```
ತಪ್ಪುಗಳು:  90 − 95 = −5,    105 − 100 = +5

MAE  = ( |−5| + |5| ) / 2      = 10 / 2   = 5
MSE  = ( 25 + 25 ) / 2         = 25
RMSE = √25                     = 5
MSLE = ( (ln91 − ln96)² + (ln106 − ln101)² ) / 2 = 0.0026
```

MAE ಮತ್ತು MSE ಯ ವ್ಯತ್ಯಾಸ — ಒಂದು ದೊಡ್ಡ ತಪ್ಪು ಬಂದರೆ?

ನಾಲ್ಕು ಮನೆಗಳ ಬೆಲೆ ಊಹೆಯ ತಪ್ಪುಗಳು (ಲಕ್ಷಗಳಲ್ಲಿ): 1, 1, 1 ಮತ್ತು ಒಂದು ದೊಡ್ಡ ತಪ್ಪು 10.

```
MAE  = (1 + 1 + 1 + 10) / 4          = 13 / 4   = 3.25
MSE  = (1 + 1 + 1 + 100) / 4         = 103 / 4  = 25.75
RMSE = √25.75                        = 5.07
```

ಒಂದೇ ದೊಡ್ಡ ತಪ್ಪು (10) MSE ಯನ್ನು ಬಹಳ ಏರಿಸಿತು (25.75), MAE ಯನ್ನು ಸ್ವಲ್ಪ ಮಾತ್ರ (3.25). ಹಾಗಾಗಿ ದೊಡ್ಡ ತಪ್ಪುಗಳನ್ನು ಕಠಿಣವಾಗಿ ಶಿಕ್ಷಿಸಬೇಕಾದರೆ MSE; ಅಸಾಮಾನ್ಯ (outlier) ಮೌಲ್ಯಗಳಿಗೆ ಹೆದರದ ಅಳತೆ ಬೇಕಾದರೆ MAE. ಎರಡರ ಅನುಕೂಲ ಬೇಕಾದರೆ **Huber loss** (ಚಿಕ್ಕ ತಪ್ಪಿಗೆ MSE ಯಂತೆ, ದೊಡ್ಡ ತಪ್ಪಿಗೆ MAE ಯಂತೆ) ಅಥವಾ **Log-Cosh** (MSE ಗೆ ಮೃದು ಪರ್ಯಾಯ) ಬಳಸುತ್ತಾರೆ.

೮. ಶೇಕಡಾ ತಪ್ಪುಗಳು: MPE ಮತ್ತು MAPE

ಏಕೆ ಶೇಕಡಾ?

₹10 ಲಕ್ಷದ ಮನೆಯ ಬೆಲೆಯಲ್ಲಿ ₹1 ಲಕ್ಷ ತಪ್ಪು = 10% (ದೊಡ್ಡ ತಪ್ಪು!). ₹1 ಕೋಟಿ (100 ಲಕ್ಷ) ಮನೆಯಲ್ಲಿ ಅದೇ ₹1 ಲಕ್ಷ ತಪ್ಪು = ಕೇವಲ 1%. ತಪ್ಪಿನ ಗಾತ್ರವನ್ನು ನಿಜವಾದ ಮೌಲ್ಯಕ್ಕೆ ಹೋಲಿಸಿದರೆ ನ್ಯಾಯ.

```
MPE  = (100%/n) Σ (y − ŷ) / y          ← ಚಿಹ್ನೆ ಉಳಿಯುತ್ತದೆ (ಧನ/ಋಣ ದಿಕ್ಕು ತಿಳಿಸುತ್ತದೆ)
MAPE = (100%/n) Σ | (y − ŷ) / y |      ← ಚಿಹ್ನೆ ತೆಗೆದು ಸರಾಸರಿ  (ಷರತ್ತು: y ≠ 0)
```

ಅದೇ IQ ಉದಾಹರಣೆ

```
ವ್ಯಕ್ತಿ 1:  (90 − 95) / 90   = −0.0556  →  5.56%
ವ್ಯಕ್ತಿ 2:  (105 − 100) / 105 = +0.0476  →  4.76%

MAPE = (5.56% + 4.76%) / 2 = 5.16%
MPE  = (−5.56% + 4.76%) / 2 = −0.40%
```

MPE ಯಲ್ಲಿ ಧನ-ಋಣ ಒಂದನ್ನೊಂದು ಸ್ವಲ್ಪ ರದ್ದು ಮಾಡಿ −0.40% ಬಂತು; ಅದರ ಚಿಹ್ನೆ ಮಾದರಿ ಒಟ್ಟಾರೆ ಯಾವ ದಿಕ್ಕಿಗೆ ಪಕ್ಷಪಾತಿ ಎಂದು ತೋರಿಸುತ್ತದೆ (ಇಲ್ಲಿ ಸ್ವಲ್ಪ ಹೆಚ್ಚು ಊಹಿಸುತ್ತಿದೆ). MAPE ತಪ್ಪಿನ ಒಟ್ಟು ಗಾತ್ರ ತೋರಿಸುತ್ತದೆ.

MAPE ಅಸಮಪಾರ್ಶ್ವ!

```
ಊಹೆ ಕಡಿಮೆ:  y = 20, ŷ = 10 →  |20 − 10| / 20 = 0.5  → MAPE = 50%
ಊಹೆ ಹೆಚ್ಚು:  y = 10, ŷ = 20 →  |10 − 20| / 10 = 1.0  → MAPE = 100%
```

ಅದೇ 10 ರ ಅಂತರ, ಆದರೆ ಹೆಚ್ಚು ಊಹಿಸಿದರೆ ಶಿಕ್ಷೆ ಎರಡು ಪಟ್ಟು. ಜೊತೆಗೆ ನಿಜವಾದ ಮೌಲ್ಯ 0 ಆದರೆ ಭಾಗಾಕಾರ ಸಾಧ್ಯವಿಲ್ಲ.

೯. Entropy ಮತ್ತು Cross-Entropy (ವರ್ಗೀಕರಣದ loss)

Entropy — "ಅನಿಶ್ಚಿತತೆ"ಯ ಅಳತೆ

ಕ್ಲೌಡ್ ಶಾನನ್ (Claude Shannon) ಸೂತ್ರ: $H(S) = -\Sigma\ p_i \cdot \log_2(p_i)$ — ಇಲ್ಲಿ p_i = ವರ್ಗ i ರ ಅನುಪಾತ. ಹೆಚ್ಚು ಅನಿಶ್ಚಿತತೆ (ಮಿಶ್ರಣ) ಇದ್ದಷ್ಟೂ entropy ಹೆಚ್ಚು.

ಉದಾಹರಣೆ: ನಾಣ್ಯ ಚಿಮ್ಮುವುದು

```
ಸರಿಯಾದ ನಾಣ್ಯ   (0.5, 0.5):  H = −(0.5·log2 0.5 + 0.5·log2 0.5) = 1.0   (ಗರಿಷ್ಠ ಅನಿಶ್ಚಿತ)
ಓರೆ ನಾಣ್ಯ       (0.9, 0.1):  H = 0.469
(ಬಹುತೇಕ ಊಹಿಸಬಹುದು)
ಯಾವಾಗಲೂ ತಲೆ   (1.0, 0.0):  H = 0
(ಸಂಪೂರ್ಣ ಖಚಿತ)
```

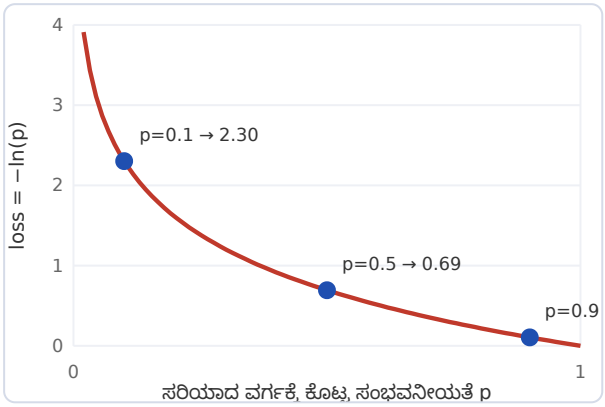
ಕ್ಲಾಸ್‌ನಲ್ಲಿ 10 ವಿದ್ಯಾರ್ಥಿಗಳು: 5 ಪಾಸ್, 5 ಫೇಲ್ ಆದರೆ H = 1 (ಅತಿ ಮಿಶ್ರ); 9 ಪಾಸ್, 1 ಫೇಲ್ ಆದರೆ H = 0.469 (ಹೆಚ್ಚು ಶುದ್ಧ).

Cross-Entropy Loss — ಊಹೆ ಮತ್ತು ಸತ್ಯದ ಹೋಲಿಕೆ

ವರ್ಗೀಕರಣದಲ್ಲಿ ಅತಿ ಹೆಚ್ಚು ಬಳಕೆ. ನೆಟ್‌ವರ್ಕ್ ಕೊಟ್ಟ ಸಂಭವನೀಯತೆಗಳನ್ನು ನಿಜವಾದ ವರ್ಗದೊಂದಿಗೆ ಹೋಲಿಸುತ್ತದೆ.

```
Binary Cross-Entropy:   L = − (1/N) Σ [ y·log(p) + (1 − y)·log(1 − p) ]
Multi-class:            L = − (1/N) Σ_i Σ_j  y(i,j) · log p(i,j)

N = ಉದಾಹರಣೆಗಳ ಸಂಖ್ಯೆ,  y = ನಿಜವಾದ ಲೇಬಲ್ (0/1),  p = "ವರ್ಗ 1" ಎಂಬ ಊಹೆಯ ಸಂಭವನೀಯತೆ,  C = ವರ್ಗಗಳ ಸಂಖ್ಯೆ
```



ಚಿತ್ರ ೨: ಸರಿಯಾದ ವರ್ಗಕ್ಕೆ ನೀಡಿದ ಸಂಭವನೀಯತೆ ಕಡಿಮೆಯಾದಷ್ಟೂ loss ಅತಿ ವೇಗವಾಗಿ ಏರುತ್ತದೆ

Binary Cross-Entropy ಲೆಕ್ಕ: ಇಮೇಲ್ ಸ್ಪ್ಯಾಮ್?

ಮೂರು ಇಮೇಲ್‌ಗಳು. y = 1 (ಸ್ಪ್ಯಾಮ್), y = 0 (ಸ್ಪ್ಯಾಮ್ ಅಲ್ಲ). p = ನೆಟ್‌ವರ್ಕ್ ಹೇಳಿದ "ಸ್ಪ್ಯಾಮ್ ಸಂಭವನೀಯತೆ".

ಇಮೇಲ್	y	p	ಲೆಕ್ಕ	loss
೧ (ಒಳ್ಳೆಯ ಊಹೆ)	1	0.9	−ln(0.9)	0.105
೨ (ಒಳ್ಳೆಯ ಊಹೆ)	0	0.2	−ln(1 − 0.2) = −ln(0.8)	0.223
೩ (ಕೆಟ್ಟ ಊಹೆ)	1	0.3	−ln(0.3)	1.204

ಸರಾಸರಿ loss = (0.105 + 0.223 + 1.204) / 3 = 0.511. ಮೂರನೆಯ ಇಮೇಲ್ (ತಪ್ಪು ಊಹೆ) ಬಹುತೇಕ ಎಲ್ಲ loss ಅನ್ನು ಕೊಡುಗೆ ನೀಡಿತು. ನೆಟ್‌ವರ್ಕ್ ಆತ್ಮವಿಶ್ವಾಸದಿಂದ ತಪ್ಪು ಹೇಳಿದರೆ (ಉದಾ: ಸ್ಪ್ಯಾಮ್‌ಗೆ p = 0.1 → loss 2.30) ಶಿಕ್ಷೆ ಇನ್ನೂ ಹೆಚ್ಚು.

Multi-class ಲೆಕ್ಕ: ಪ್ರಾಣಿ ಚಿತ್ರ (ಸತ್ಯ = ನಾಯಿ)

ಸತ್ಯ ಲೇಬಲ್ one-hot: [ನಾಯಿ 1, ಮೊಲ 0, ಬೆಕ್ಕು 0]. ಒಂದೇ ವರ್ಗದ y = 1, ಉಳಿದವು 0 — ಹಾಗಾಗಿ ಸೂತ್ರದಲ್ಲಿ ಉಳಿಯುವುದು **−ln(ಸರಿಯಾದ ವರ್ಗದ p)** ಮಾತ್ರ.

```
ಭಾಗ ೫ ರ softmax: [0.687, 0.187, 0.126]   →  loss = −ln(0.687) = 0.375
ಕೆಟ್ಟ ನೆಟ್‌ವರ್ಕ್:  [0.05, 0.90, 0.05]    →  loss = −ln(0.05)  = 3.00
```

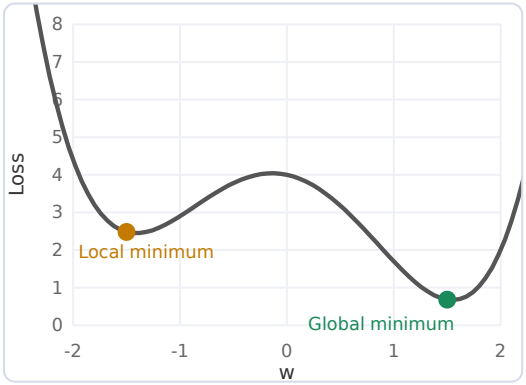
ಇತರ ವಿಶೇಷ loss ಗಳು (ಪರಿಚಯ)

Loss	ಯಾವಾಗ ಬಳಸುತ್ತಾರೆ?
Hinge / Squared Hinge	ಎರಡು ವರ್ಗಗಳ ನಡುವೆ ಹೆಚ್ಚು ಅಂತರ (margin) ಬೇಕಾದಾಗ (SVM ಶೈಲಿ)
Focal Loss	ಒಂದು ವರ್ಗ ಅತಿ ಅಪರೂಪವಾದಾಗ (ಉದಾ: 1000 ವ್ಯವಹಾರಗಳಲ್ಲಿ 1 ವಂಚನೆ) — ಸುಲಭ ಉದಾಹರಣೆಗಳ ತೂಕ ಕಡಿಮೆ, ಕಷ್ಟದ್ದಕ್ಕೆ ಹೆಚ್ಚು
KL Divergence	ಎರಡು ಸಂಭವನೀಯತೆ ಹಂಚಿಕೆಗಳು ಎಷ್ಟು ಬೇರೆ ಎಂದು ಅಳೆಯಲು
CTC Loss	ಧ್ವನಿ → ಪಠ್ಯ ಗುರುತಿಸುವಿಕೆ (ಕಾಲ-ಹೊಂದಾಣಿಕೆ ಗೊತ್ತಿಲ್ಲದಾಗ)
Dice / Jaccard / Tversky	ವೈದ್ಯಕೀಯ ಸ್ಕ್ಯಾನ್‌ನಲ್ಲಿ ಗೆಡ್ಡೆಯ ಭಾಗ ಗುರುತಿಸುವ (segmentation) ಕೆಲಸ; ಎರಡು ಪ್ರದೇಶಗಳ ಅತಿಕ್ರಮಣ ಅಳೆಯುತ್ತವೆ

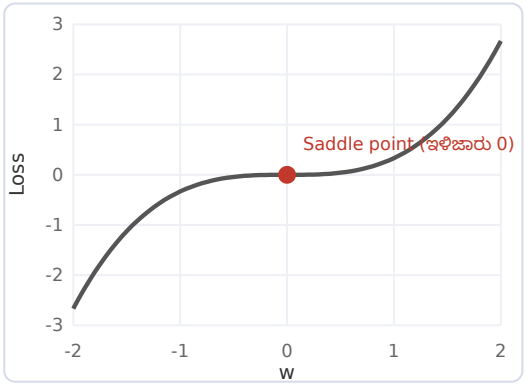
೧೦. Gradient Descent ನ ಸಮಸ್ಯೆಗಳು: Local minimum, Saddle point, Overfitting

Gradient Descent ಎಂದರೆ loss ಕಡಿಮೆ ಮಾಡಲು ತೂಕಗಳನ್ನು ಗ್ರೇಡಿಯಂಟ್‌ನ ವಿರುದ್ಧ ದಿಕ್ಕಿಗೆ ಸಣ್ಣ ಹೆಜ್ಜೆಯಿಂದ ಬದಲಿಸುವುದು. ಹೆಜ್ಜೆಯ ಗಾತ್ರ = **learning rate (η)**. ಆದರೆ ಇದಕ್ಕೆ ಕೆಲವು ಅಡ್ಡಿಗಳಿವೆ:

ಬಗೆ	ವಿವರ
Convex ಸಮಸ್ಯೆ	ಒಂದೇ global minimum, ಮೇಲ್ಮೈ ಬಟ್ಟಲಿನಂತೆ ನಯ → gradient descent ಯಾವಾಗಲೂ ಅತ್ಯುತ್ತಮ ಉತ್ತರ ಕಂಡುಹಿಡಿಯುತ್ತದೆ.
Non-convex ಸಮಸ್ಯೆ	ಹಲವು local minima ಮತ್ತು saddle points; ಮೇಲ್ಮೈ ಅಸಮ → ಅತ್ಯುತ್ತಮ ಉತ್ತರ ತಲುಪುವ ಮೊದಲೇ ಸಿಕ್ಕಿಕೊಳ್ಳಬಹುದು. ನಿಜವಾದ ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ಗಳು ಇದೇ ಬಗೆ.



ಚಿತ್ರ ೩: Local minimum — ಕಣಿವೆಯಂತೆ ಕಾಣುತ್ತದೆ ಆದರೆ ಅತಿ ಕೆಳಗಿನದಲ್ಲ



ಚಿತ್ರ ೪: Saddle point — ಗ್ರೇಡಿಯಂಟ್ 0, ಆದರೆ ಕನಿಷ್ಠವಲ್ಲ

- **Local minimum:** ಗ್ರೇಡಿಯಂಟ್ = 0, ಆದರೆ ಇದು ಅತ್ಯುತ್ತಮ ಪರಿಹಾರವಲ್ಲ; ಮಾದರಿ ಇಲ್ಲೇ ಸಿಕ್ಕಿಕೊಳ್ಳಬಹುದು. *ಹೋಲಿಕೆ:* ಬೆಟ್ಟದ ನಡುವಿನ ಒಂದು ಹೊಂಡದಲ್ಲಿ ನೀರು ನಿಂತಂತೆ — ಊರಿನ ಕೆರೆಗೆ ಹರಿಯಲಿಲ್ಲ.
- **Saddle point:** ಗ್ರೇಡಿಯಂಟ್ = 0, ಆದರೆ ಒಂದು ಬದಿ ಏರುತ್ತದೆ, ಇನ್ನೊಂದು ಇಳಿಯುತ್ತದೆ (ಕುದುರೆ ಸವಾರಿ ಜೀನಿನಂತೆ). ಮಾದರಿ ಬೇಗನೇ ನಿಲ್ಲಬಹುದು.

Overfitting ಮತ್ತು Underfitting

ಸಮಸ್ಯೆ	ಕಾರಣ	ಲಕ್ಷಣ
Underfitting	ಮಾದರಿ ತುಂಬ ಸರಳ	ವರ್ಗಗಳನ್ನು ಬೇರ್ಪಡಿಸಲಾಗುವುದಿಲ್ಲ; **ಎಲ್ಲ ದತ್ತಾಂಶದಲ್ಲೂ** ಹೆಚ್ಚು ತಪ್ಪು
Overfitting	ಮಾದರಿ ತುಂಬ ಸಂಕೀರ್ಣ	ತರಬೇತಿ ದತ್ತಾಂಶವನ್ನು (ಅದರ noise ಸಹಿತ) ಪರಿಪೂರ್ಣವಾಗಿ ಕಲಿಯುತ್ತದೆ, ಆದರೆ ಹೊಸ ದತ್ತಾಂಶದಲ್ಲಿ ವಿಫಲ

> **ಕಥೆ: ಉರು ಹೊಡೆಯುವ ವಿದ್ಯಾರ್ಥಿ**
>
> ಒಬ್ಬ ವಿದ್ಯಾರ್ಥಿ ಕಳೆದ ವರ್ಷದ ಪ್ರಶ್ನೆಪತ್ರಿಕೆಯ ಉತ್ತರಗಳನ್ನು ಅಕ್ಷರಶಃ ಉರು ಹೊಡೆದಿದ್ದಾನೆ. ಅದೇ ಪ್ರಶ್ನೆ ಬಂದರೆ 100/100! ಆದರೆ ಸ್ವಲ್ಪ ಬದಲಾದ ಪ್ರಶ್ನೆ ಬಂದರೆ ಫೇಲ್. ಇದು **overfitting**. ಪಾಠವನ್ನೇ ಓದದ ವಿದ್ಯಾರ್ಥಿ ಎಲ್ಲದರಲ್ಲೂ ಫೇಲ್ — ಅದು **underfitting**. ಒಳ್ಳೆಯ ವಿದ್ಯಾರ್ಥಿ ಪರಿಕಲ್ಪನೆ ಅರ್ಥಮಾಡಿಕೊಂಡು ಹೊಸ ಪ್ರಶ್ನೆಯಲ್ಲೂ ಗೆಲ್ಲುತ್ತಾನೆ.

೧೧. Batch Gradient Descent

Batch GD: ಪ್ರತಿ ಹೆಜ್ಜೆಯಲ್ಲಿ **ಇಡೀ ತರಬೇತಿ ದತ್ತಾಂಶ** ಬಳಸಿ ಸರಾಸರಿ ಗ್ರೇಡಿಯಂಟ್ ಲೆಕ್ಕ ಹಾಕಿ, ಒಂದೇ ಬಾರಿ ತೂಕ ಬದಲಿಸುತ್ತದೆ.

```
θ = θ − η · ∇L(θ)          θ = ಮಾದರಿಯ ತೂಕಗಳು,  η = learning rate,  ∇L(θ) = ಇಡೀ ದತ್ತಾಂಶದ ಗ್ರೇಡಿಯಂಟ್
```

ಅನುಕೂಲ (+)	ಅನಾನುಕೂಲ (−)
ಸ್ಥಿರ ಒಮ್ಮುಖತೆ: ಎಲ್ಲ ಉದಾಹರಣೆಗಳ ಸರಾಸರಿ — ಕಡಿಮೆ noise. **ಸಮಗ್ರ ನೋಟ:** ಇಡೀ ದತ್ತಾಂಶದ ದೃಷ್ಟಿಕೋನ.	**ದುಬಾರಿ ಲೆಕ್ಕಾಚಾರ:** ಪ್ರತಿ ಹೆಜ್ಜೆಗೂ ಇಡೀ ದತ್ತಾಂಶ — ನಿಧಾನ. **ಮೆಮೊರಿ:** ಎಲ್ಲವನ್ನೂ ಮೆಮೊರಿಯಲ್ಲಿ ಇಡಬೇಕು — ದೊಡ್ಡ ದತ್ತಾಂಶಕ್ಕೆ ಅಸಾಧ್ಯ.

> **ಹೋಲಿಕೆ**
>
> ಇಡೀ ಕ್ಲಾಸಿನ ಎಲ್ಲ 1000 ವಿದ್ಯಾರ್ಥಿಗಳ ಉತ್ತರಪತ್ರಿಕೆ ತಿದ್ದಿ, ಸರಾಸರಿ ಆಧರಿಸಿ ಒಮ್ಮೆ ಮಾತ್ರ ಬೋಧನಾ ವಿಧಾನ ಬದಲಿಸುವುದು: ನಿಖರ, ಆದರೆ ತುಂಬ ನಿಧಾನ.

೧೨. Stochastic ಮತ್ತು Mini-Batch Gradient Descent

Stochastic Gradient Descent (SGD)

ಪ್ರತಿ ಹೆಜ್ಜೆಗೆ **ಕೇವಲ ಒಂದು** ಉದಾಹರಣೆ ಬಳಸಿ ತೂಕ ಬದಲಿಸುತ್ತದೆ: $\theta = \theta - \eta \cdot \nabla L(\theta; x^{(i)}, y^{(i)})$.

ಅನುಕೂಲ (+)	ಅನಾನುಕೂಲ (−)
ಆಗಾಗ್ಗೆ ಬದಲಾವಣೆ → ಬೇಗ ಕಲಿಯಬಲ್ಲದು ಕಡಿಮೆ ಮೆಮೊರಿ ಹರಿವಿನ ದತ್ತಾಂಶಕ್ಕೆ (online learning) ಸೂಕ್ತ Local minimum ದಿಂದ ಹೊರಬರಲು ಸಹಾಯ	Noisy (ಅಲುಗಾಡುವ) ದಾರಿ ಹೆಚ್ಚು learning rate ಇದ್ದರೆ ಕನಿಷ್ಠ ದಾಟಿ ಹೋಗಬಹುದು learning rate ಬಗ್ಗೆ ಸೂಕ್ಷ್ಮ

Mini-Batch Gradient Descent

ಮಧ್ಯದ ದಾರಿ: ಇಡೀ ದತ್ತಾಂಶವೂ ಅಲ್ಲ, ಒಂದೇ ಉದಾಹರಣೆಯೂ ಅಲ್ಲ — **ಸಣ್ಣ ಯಾದೃಚ್ಛಿಕ ಗುಂಪು (mini-batch, ಉದಾ: 32, 64 ಅಥವಾ 128 ಉದಾಹರಣೆಗಳು)**: $\theta = \theta - \eta \cdot \nabla L(\theta; \{x^{(i)}, y^{(i)}\}, i=1..m)$.

ಅನುಕೂಲ (+)	ಅನಾನುಕೂಲ (−)
SGD ಯ noise ಮತ್ತು Batch ನ ಸ್ಥಿರತೆಯ ಸಮತೋಲನ ಕಡಿಮೆ ಮೆಮೊರಿ GPU/ಸಮಾಂತರ ಲೆಕ್ಕಾಚಾರಕ್ಕೆ ಸೂಕ್ತ	Batch size ಮತ್ತು learning rate ಟ್ಯೂನ್ ಮಾಡಬೇಕು Batch ಗಿಂತ ಕಡಿಮೆ ಸ್ಥಿರ (size ಚಿಕ್ಕದಿದ್ದರೆ)

ಒಂದೇ ಉದಾಹರಣೆ, ಮೂರು ವಿಧಾನ — ನಿಜವಾದ ಲೆಕ್ಕ

ಸಮಸ್ಯೆ: y = 2x ಎಂಬ ನಿಯಮ ಕಲಿಯುವುದು (ಒಂದೇ ತೂಕ w; ಗುರಿ w = 2)

ದತ್ತಾಂಶ: (x, y) = (1, 2), (2, 4), (3, 6). ಊಹೆ $\hat{y} = w \cdot x$. Loss = $(w \cdot x - y)^2$. ಒಂದು ಉದಾಹರಣೆಯ ಗ್ರೇಡಿಯಂಟ್ = $2(w \cdot x - y) \cdot x$. ಆರಂಭ w = 0, $\eta = 0.05$. **ಒಂದು ಪೂರ್ಣ ಸುತ್ತು (epoch)** ನಂತರ w ಎಷ್ಟು?

ವಿಧಾನ	ಹೆಜ್ಜೆಗಳು	w ಬದಲಾವಣೆ	1 epoch ನಂತರ w
Batch	1 ಹೆಜ್ಜೆ	ಗ್ರೇಡಿಯಂಟ್‌ಗಳು (−4, −16, −36) ರ ಸರಾಸರಿ = −18.67 w = 0 − 0.05×(−18.67) = 0.933	**0.933**
SGD (ಒಂದೊಂದು)	3 ಹೆಜ್ಜೆಗಳು	(1,2): g = −4 → w = 0.2 (2,4): g = 2(0.4−4)·2 = −14.4 → w = 0.92 (3,6): g = 2(2.76−6)·3 = −19.44 → w = 1.892	**1.892**
Mini-batch (ಗಾತ್ರ 2)	2 ಹೆಜ್ಜೆಗಳು	{(1,2),(2,4)}: ಸರಾಸರಿ g = (−4−16)/2 = −10 → w = 0.5 {(3,6)}: g = 2(1.5−6)·3 = −27 → w = 1.85	**1.85**

ಒಂದು epoch ನಲ್ಲಿ SGD (1.892) ಮತ್ತು mini-batch (1.85) ಗುರಿ 2 ರ ಹತ್ತಿರ ಬಂದವು; Batch ಕೇವಲ 0.933. ಏಕೆಂದರೆ ಅವು ಹೆಚ್ಚು ಬಾರಿ ತಿದ್ದಿಕೊಂಡವು. (Batch ಗೆ ಇದೇ ದೂರ ತಲುಪಲು ಸುಮಾರು 5 epochs ಬೇಕು: 0.93 → 1.43 → 1.70 → 1.84 → 1.91.)

ಒಂದು epoch ನಲ್ಲಿ ಎಷ್ಟು ಬಾರಿ ತೂಕ ಬದಲಾಗುತ್ತದೆ?

ದತ್ತಾಂಶದಲ್ಲಿ 1000 ಉದಾಹರಣೆಗಳಿವೆ ಎಂದುಕೊಳ್ಳಿ:

ವಿಧಾನ	ಪ್ರತಿ ಹೆಜ್ಜೆಗೆ ಬಳಸುವ ಉದಾಹರಣೆಗಳು	1 epoch ನಲ್ಲಿ ತೂಕ ಬದಲಾವಣೆಗಳು
Batch	1000	1
Mini-batch (32)	32	$\lceil 1000/32 \rceil = 32$
SGD	1	1000

Batch ಮತ್ತು SGD ಹೋಲಿಕೆ

ಅಂಶ	Batch GD	SGD
ದತ್ತಾಂಶ ಬಳಕೆ	ಇಡೀ ದತ್ತಾಂಶ	ಪ್ರತಿ ಹೆಜ್ಜೆಗೆ ಒಂದು ಉದಾಹರಣೆ
ವೇಗ	ನಿಧಾನ	ಆಗಾಗ್ಗೆ ಬದಲಾವಣೆಯಿಂದ ವೇಗ
ಗ್ರೇಡಿಯಂಟ್ ನಿಖರತೆ	ಹೆಚ್ಚು ನಿಖರ	Noisy, ಕಡಿಮೆ ನಿಖರ
ಮೆಮೊರಿ / ಲೆಕ್ಕಾಚಾರ	ಹೆಚ್ಚು	ಕಡಿಮೆ
Local minima	ಸಿಕ್ಕಿಕೊಳ್ಳಬಹುದು	ಹೊರಬರಬಲ್ಲದು
ದೊಡ್ಡ ದತ್ತಾಂಶ	ಸೂಕ್ತವಲ್ಲ	ಸೂಕ್ತ
ಸ್ವಭಾವ	Deterministic (ಪ್ರತಿ ಬಾರಿ ಒಂದೇ ಫಲಿತಾಂಶ)	Stochastic (ಪ್ರತಿ ಬಾರಿ ಬೇರೆ)
Learning rate	ಸಾಮಾನ್ಯವಾಗಿ ಸ್ಥಿರ	ಕ್ರಮೇಣ ಬದಲಿಸಬಹುದು
Shuffle ಬೇಕೇ?	ಬೇಡ	ಪ್ರತಿ epoch ನಲ್ಲಿ ದತ್ತಾಂಶ ಕಲಸಬೇಕು
ಅಂತಿಮ ಫಲಿತಾಂಶ	Convex ಸಮಸ್ಯೆಯಲ್ಲಿ global minimum ತಲುಪುವ ಸಾಧ್ಯತೆ ಹೆಚ್ಚು	Local minimum / saddle point ನಲ್ಲಿ ನಿಲ್ಲಬಹುದು (ಆದರೆ noise ನಿಂದ ತಪ್ಪಿಸಿಕೊಳ್ಳಲೂಬಹುದು)

> **ಹೋಲಿಕೆ: ಮೂರು ಶಿಕ್ಷಕರು**
>
> **Batch** ಶಿಕ್ಷಕ: ಇಡೀ ಕ್ಲಾಸಿನ ಉತ್ತರಪತ್ರಿಕೆ ತಿದ್ದಿ ಒಮ್ಮೆ ಮಾತ್ರ ಪಾಠ ಬದಲಿಸುತ್ತಾರೆ — ನಿಖರ ಆದರೆ ನಿಧಾನ. **SGD** ಶಿಕ್ಷಕ: ಒಬ್ಬ ವಿದ್ಯಾರ್ಥಿಯ ತಪ್ಪು ನೋಡಿದ ತಕ್ಷಣ ಪಾಠ ಬದಲಿಸುತ್ತಾರೆ — ವೇಗ ಆದರೆ ಚಂಚಲ. **Mini-batch** ಶಿಕ್ಷಕ: ಒಂದು ಬೆಂಚ್‌ನ (32 ವಿದ್ಯಾರ್ಥಿಗಳ) ಉತ್ತರ ನೋಡಿ ಬದಲಿಸುತ್ತಾರೆ — ಎರಡರ ಉತ್ತಮ ಸಮತೋಲನ. ಇಂದು ಹೆಚ್ಚಿನ ಡೀಪ್ ಲರ್ನಿಂಗ್‌ನಲ್ಲಿ ಬಳಸುವುದು ಇದನ್ನೇ.

೧೩. Momentum ಮತ್ತು Nesterov (NAG)

Momentum

ಭೌತಶಾಸ್ತ್ರದಿಂದ ಬಂದ ಕಲ್ಪನೆ: ವಸ್ತುವಿನ ಚಲನೆ ಈಗಿನ ಬಲದ ಮೇಲೆ ಮಾತ್ರವಲ್ಲ, ಹಿಂದಿನ ವೇಗದ ಮೇಲೂ ಅವಲಂಬಿತ. ಹಿಂದಿನ ಗ್ರೇಡಿಯಂಟ್‌ಗಳನ್ನು "ನೆನಪಿಟ್ಟುಕೊಂಡು" ದಾರಿಯನ್ನು ನಯಗೊಳಿಸುತ್ತದೆ, ಅಲುಗಾಟ ಕಡಿಮೆ ಮಾಡುತ್ತದೆ.

```
v_t     = γ · v_(t−1) + η · ∇L(θ_t)
θ_(t+1) = θ_t − v_t

v = ವೇಗ (velocity),  γ = momentum ಗುಣಾಂಕ (0 ರಿಂದ 1, ಸಾಮಾನ್ಯವಾಗಿ ≈ 0.9),  η = learning rate
```

ಹೋಲಿಕೆ: ಇಳಿಜಾರಿನಲ್ಲಿ ಉರುಳುವ ಚೆಂಡು

ಘಾಟಿಯಲ್ಲಿ ಇಳಿಯುವ ಬೈಕ್ ಒಂದೊಂದು ಕ್ಷಣದ ಇಳಿಜಾರನ್ನು ಮಾತ್ರ ನೋಡುವುದಿಲ್ಲ — ಸಂಗ್ರಹವಾದ ವೇಗದಿಂದ ಚಿಕ್ಕ ಗುಂಡಿಗಳನ್ನೂ ದಾಟಿ ಮುಂದೆ ಹೋಗುತ್ತದೆ. Momentum ಸಹ ಅದೇ: ಸ್ಥಿರ ದಿಕ್ಕಿನಲ್ಲಿ ವೇಗ ಹೆಚ್ಚಿಸಿಕೊಳ್ಳುತ್ತದೆ, ಅಲುಗಾಡುವ ದಿಕ್ಕಿನಲ್ಲಿ ತಗ್ಗಿಸುತ್ತದೆ.

ಲೆಕ್ಕ: $E(w) = (w - 3)^2$ ನಲ್ಲಿ ಸಾಮಾನ್ಯ GD vs Momentum

ಆರಂಭ $w = 0$, $\eta = 0.1$. ಗ್ರೇಡಿಯಂಟ್ $= 2(w - 3)$. Momentum ಗೆ $\gamma = 0.9$. (ಗುರಿ $w = 3$.)

ಹೆಜ್ಜೆ	1	2	3	4	5	6
ಸಾಮಾನ್ಯ GD ($\gamma = 0$)	0.60	1.08	1.46	1.77	2.02	2.21
Momentum ($\gamma = 0.9$)	0.60	1.62	2.81	3.93	4.74	5.13

Momentum ತುಂಬ ವೇಗವಾಗಿ ಗುರಿಯ ಕಡೆ ಹೋಯಿತು (ಹೆಜ್ಜೆ 3 ರಲ್ಲೇ 2.81). ಆದರೆ ಸಂಗ್ರಹವಾದ ವೇಗದಿಂದ ಗುರಿಯನ್ನು **ದಾಟಿ** (3.93, 4.74, 5.13) ಹೋಗಿ, ನಂತರ ಹಿಂತಿರುಗುತ್ತದೆ (ಮುಂದಿನ ಹೆಜ್ಜೆಗಳು 5.05, 4.57 ...). ಹಾಗಾಗಿ ವೇಗ ಹೆಚ್ಚಿದಂತೆ γ ಅಥವಾ η ಅನ್ನು ಸೂಕ್ತವಾಗಿ ಇಡುವುದು ಮುಖ್ಯ.

```
from tensorflow.keras.optimizers import SGD
optimizer = SGD(learning_rate=0.01, momentum=0.9)
```

Nesterov Accelerated Gradient (NAG)

Momentum ನ ಸುಧಾರಿತ ರೂಪ: **ಮುಂದೆ ಒಮ್ಮೆ ನೋಡಿ** (look-ahead) ನಂತರ ಹೆಜ್ಜೆ ಇಡುತ್ತದೆ. ಈಗಿನ ಸ್ಥಾನ θ_t ದಲ್ಲಿ ಅಲ್ಲ, ಅಂದಾಜು ಮುಂದಿನ ಸ್ಥಾನ $\boldsymbol{\theta_t - \gamma \cdot v_{t-1}}$ ದಲ್ಲಿ ಗ್ರೇಡಿಯಂಟ್ ಲೆಕ್ಕ ಹಾಕುತ್ತದೆ:

```
v_t     = γ · v_(t−1) + η · ∇L( θ_t − γ · v_(t−1) )
θ_(t+1) = θ_t − v_t
```

ಹೋಲಿಕೆ

ಬೈಕ್‌ನಲ್ಲಿ ತಿರುವು ಬರುವ ಮೊದಲೇ ಮುಂದಿನ ರಸ್ತೆ ನೋಡಿ ಬ್ರೇಕ್ ಹಾಕುವಂತೆ. ಮೂಲ Momentum ತಿರುವಿನ ಬಳಿ ಬಂದ ಮೇಲೆ ಮಾತ್ರ ಗೊತ್ತಾಗಿ ದಾಟಿ ಹೋಗುತ್ತದೆ; NAG ಮೊದಲೇ ಗಮನಿಸಿ ಹೆಚ್ಚು ಸ್ಥಿರವಾಗಿ ಹೋಗುತ್ತದೆ.

```
optimizer = SGD(learning_rate=0.01, momentum=0.9, nesterov=True)
```

೧೪. AdaGrad, RMSProp, Adadelta

ಇವು ಮೂರೂ **ಪ್ರತಿ ತೂಕಕ್ಕೆ ತನ್ನದೇ ಆದ learning rate** ಬಳಸುವ "ಹೊಂದಿಕೊಳ್ಳುವ" (adaptive) ಆಪ್ಟಿಮೈಜರ್‌ಗಳು.

AdaGrad

ಅಪರೂಪದ ವೈಶಿಷ್ಟ್ಯಗಳಿಗೆ (infrequent features) ದೊಡ್ಡ ಹೆಜ್ಜೆ, ಆಗಾಗ್ಗೆ ಬರುವವುಗಳಿಗೆ ಚಿಕ್ಕ ಹೆಜ್ಜೆ. ವಿರಳ ದತ್ತಾಂಶಕ್ಕೆ ಒಳ್ಳೆಯದು; learning rate ಕೈಯಾರೆ ಬದಲಿಸಬೇಕಿಲ್ಲ.

```
G_t     = G_(t−1) + g_t²                  (ಇದುವರೆಗಿನ ಗ್ರೇಡಿಯಂಟ್‌ಗಳ ವರ್ಗಗಳ ಮೊತ್ತ)
θ_(t+1) = θ_t − η / √(G_t + ε) · g_t       ε ≈ 1e-8 (ಸೊನ್ನೆಯಿಂದ ಭಾಗಿಸದಂತೆ)
```

ಲೆಕ್ಕ: ಎರಡು ತೂಕಗಳು, ಒಂದಕ್ಕೆ ದೊಡ್ಡ ಗ್ರೇಡಿಯಂಟ್, ಇನ್ನೊಂದಕ್ಕೆ ಚಿಕ್ಕದು

η = 0.1. ತೂಕ A ಗೆ ಗ್ರೇಡಿಯಂಟ್ ಯಾವಾಗಲೂ 4; ತೂಕ B ಗೆ ಯಾವಾಗಲೂ 0.1.

ಹೆಜ್ಜೆ	A: G ಮತ್ತು ಬದಲಾವಣೆ	B: G ಮತ್ತು ಬದಲಾವಣೆ
1	G = 16 → 0.1/4 × 4 = **0.100**	G = 0.01 → 0.1/0.1 × 0.1 = **0.100**
2	G = 32 → 0.1/5.66 × 4 = **0.071**	G = 0.02 → 0.1/0.141 × 0.1 = **0.071**
3	G = 48 → 0.1/6.93 × 4 = **0.058**	G = 0.03 → 0.1/0.173 × 0.1 = **0.058**

ಗ್ರೇಡಿಯಂಟ್ 40 ಪಟ್ಟು ಬೇರೆಯಾಗಿದ್ದರೂ ಎರಡೂ ತೂಕಗಳಿಗೆ **ಒಂದೇ ಗಾತ್ರದ ಹೆಜ್ಜೆ** ಸಿಕ್ಕಿತು — ಇದೇ AdaGrad ನ ಜಾದೂ. ಆದರೆ ಸಮಸ್ಯೆ: G ಯಾವಾಗಲೂ ಹೆಚ್ಚುತ್ತಲೇ ಇರುವುದರಿಂದ ಹೆಜ್ಜೆ ಕುಗ್ಗುತ್ತಲೇ ಹೋಗುತ್ತದೆ (0.100 → 0.071 → 0.058 …), ಕೊನೆಗೆ ಕಲಿಕೆ ನಿಂತೇ ಹೋಗಬಹುದು.

```
from tensorflow.keras.optimizers import Adagrad
optimizer = Adagrad(learning_rate=0.01, epsilon=1e-7)
```

RMSProp

AdaGrad ನ ವೇಗವಾಗಿ ಕುಗ್ಗುವ learning rate ಸಮಸ್ಯೆಗೆ ಪರಿಹಾರ: ಎಲ್ಲ ಹಳೆಯ ಗ್ರೇಡಿಯಂಟ್‌ಗಳ ಮೊತ್ತದ ಬದಲು **ಘಾತೀಯವಾಗಿ ಮರೆಯುವ ಸರಾಸರಿ** (exponentially decaying average) ಬಳಸುತ್ತದೆ. ಹಳೆಯದು ಕ್ರಮೇಣ ಮರೆತು ಹೋಗುವುದರಿಂದ ಹೆಚ್ಚು ಹೆಜ್ಜೆಗಳ ನಂತರವೂ ಕಲಿಕೆ ಮುಂದುವರಿಯುತ್ತದೆ. Adam ನ ಆಧಾರ ಇದೇ.

```
v_t     = β2 · v_(t−1) + (1 − β2) · g_t²          β2 ≈ 0.9
θ_(t+1) = θ_t − η / √(v_t + ε) · g_t
```

ಲೆಕ್ಕ (g = 2 ಸ್ಥಿರ, η = 0.01, β2 = 0.9, v_0 = 0)

```
ಹೆಜ್ಜೆ 1: v = 0.9×0   + 0.1×4 = 0.400  → ಹೆಜ್ಜೆ = 0.01/√0.400 × 2 = 0.0316
ಹೆಜ್ಜೆ 2: v = 0.9×0.4 + 0.1×4 = 0.760  → ಹೆಜ್ಜೆ = 0.01/√0.760 × 2 = 0.0229
ಹೆಜ್ಜೆ 3: v = 0.9×0.76+ 0.1×4 = 1.084  → ಹೆಜ್ಜೆ = 0.01/√1.084 × 2 = 0.0192
```

v ಕ್ರಮೇಣ 4 (= g^2) ಕ್ಕೆ ಸಮೀಪಿಸುತ್ತದೆ, ಹೆಜ್ಜೆ ≈ η ಗೆ ನೆಲೆಸುತ್ತದೆ — AdaGrad ನಂತೆ ಶೂನ್ಯಕ್ಕೆ ಕುಸಿಯುವುದಿಲ್ಲ.

```
from tensorflow.keras.optimizers import RMSprop
optimizer = RMSprop(learning_rate=0.001, rho=0.9, epsilon=1e-7)
```

Adadelta

RMSProp ನ ಸುಧಾರಣೆ: learning rate ಅನ್ನು ಕೈಯಾರೆ ಕೊಡುವ ಅಗತ್ಯವೇ ಇಲ್ಲ. ಹಿಂದಿನ **ಹೆಜ್ಜೆಗಳ ವರ್ಗದ** ಚಲಿಸುವ ಸರಾಸರಿ ಮತ್ತು **ಗ್ರೇಡಿಯಂಟ್‌ಗಳ ವರ್ಗದ** ಚಲಿಸುವ ಸರಾಸರಿ ಇವುಗಳ ಅನುಪಾತದಿಂದ ಹೆಜ್ಜೆ ನಿರ್ಧರಿಸುತ್ತದೆ.

```
Δθ_t     = − RMS(Δθ_(t−1)) / RMS(g_t) · g_t
θ_(t+1)  = θ_t + Δθ_t                ε ≈ 1e-6

from tensorflow.keras.optimizers import Adadelta
optimizer = Adadelta(rho=0.95, epsilon=1e-6)
```

೧೫. Adam ಮತ್ತು Nadam

Adam (Adaptive Moment Estimation)

ಡೀಪ್ ಲರ್ನಿಂಗ್‌ನಲ್ಲಿ ಅತಿ ಹೆಚ್ಚು ಬಳಸುವ ಆಪ್ಟಿಮೈಜರ್. **Momentum + RMSProp** ಎರಡರ ಲಾಭ ಒಟ್ಟಿಗೆ. ಇದು ಎರಡು ಚಲಿಸುವ ಸರಾಸರಿಗಳನ್ನು ಇಡುತ್ತದೆ:

- $\mathbf{m_t}$ — ಗ್ರೇಡಿಯಂಟ್‌ನ ಸರಾಸರಿ (ದಿಕ್ಕಿನ "ನೆನಪು", Momentum ನಂತೆ)
- $\mathbf{v_t}$ — ಗ್ರೇಡಿಯಂಟ್ ವರ್ಗದ ಸರಾಸರಿ (ಹೆಜ್ಜೆಯ ಗಾತ್ರ ಹೊಂದಿಸಲು, RMSProp ನಂತೆ)

```
m_t = β1·m_(t−1) + (1 − β1)·g_t              β1 = 0.9
v_t = β2·v_(t−1) + (1 − β2)·g_t²             β2 = 0.999
m̂_t = m_t / (1 − β1^t),   v̂_t = v_t / (1 − β2^t)     ← ಆರಂಭದ ಹಂತದ bias ತಿದ್ದುಪಡಿ
θ_(t+1) = θ_t − η / (√v̂_t + ε) · m̂_t         η = 0.001
```

ಲೆಕ್ಕ: ಮೊದಲ ಹೆಜ್ಜೆ (g = 0.5, $m_0 = v_0 = 0$)

```
m_1 = 0.9×0 + 0.1×0.5       = 0.05
v_1 = 0.999×0 + 0.001×0.25  = 0.00025

ತಿದ್ದುಪಡಿ ಇಲ್ಲದೆ m ಮತ್ತು v ಬಹಳ ಚಿಕ್ಕ (0 ಕ್ಕೆ ಒಲವು). ತಿದ್ದುಪಡಿ:
m̂_1 = 0.05 / (1 − 0.9)      = 0.5
v̂_1 = 0.00025 / (1 − 0.999) = 0.25

ಹೆಜ್ಜೆ = 0.001 × 0.5 / (√0.25 + 1e-8) = 0.001 × 0.5 / 0.5 = 0.001
```

ಗ್ರೇಡಿಯಂಟ್ 0.5 ಆಗಿರಲಿ, 50 ಆಗಿರಲಿ, ಮೊದಲ ಹೆಜ್ಜೆ $\approx \eta = 0.001$ — ಅಂದರೆ ಗ್ರೇಡಿಯಂಟ್‌ನ ಗಾತ್ರದಿಂದ ಹೆಜ್ಜೆ ಸ್ವತಂತ್ರ, ದಿಕ್ಕು ಮಾತ್ರ ಮುಖ್ಯ. ಅದಕ್ಕೇ Adam ಹೆಚ್ಚು ಟ್ಯೂನಿಂಗ್ ಇಲ್ಲದೆಯೇ ಚೆನ್ನಾಗಿ ಕೆಲಸ ಮಾಡುತ್ತದೆ.

ಹೋಲಿಕೆ: ಚತುರ ಜಿಪಿಎಸ್

Adam ಒಂದು ಬುದ್ಧಿವಂತ ಜಿಪಿಎಸ್‌ನಂತೆ: ಇದುವರೆಗೆ ಹೋದ ದಿಕ್ಕನ್ನು ನೆನಪಿಟ್ಟುಕೊಳ್ಳುತ್ತದೆ (m, momentum), ಮತ್ತು ರಸ್ತೆ ಎಷ್ಟು ಏರುಪೇರಾಗಿದೆ ಎಂಬುದಕ್ಕೆ ತಕ್ಕಂತೆ ವೇಗ ಹೊಂದಿಸುತ್ತದೆ (v, adaptive).

```
from tensorflow.keras.optimizers import Adam
optimizer = Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-7)
```

Nadam

Adam + **Nesterov ನ look-ahead**. ಈಗಿನ ಗ್ರೇಡಿಯಂಟ್ ಮಾತ್ರವಲ್ಲ, ಗ್ರೇಡಿಯಂಟ್ ಎಲ್ಲಿಗೆ ಹೋಗುವ ನಿರೀಕ್ಷೆ ಇದೆ ಎಂಬುದನ್ನು ಬಳಸಿ ತೂಕ ಬದಲಿಸುತ್ತದೆ — ಇನ್ನಷ್ಟು ವೇಗ ಮತ್ತು ನಯವಾದ ಕಲಿಕೆ. ಮೂರು ಕಲ್ಪನೆಗಳ ಸಂಗಮ: **Momentum + Adaptive learning rate + Nesterov look-ahead**.

```
from tensorflow.keras.optimizers import Nadam
optimizer = Nadam(learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-7)
```

೧೬. ಯಾವುದನ್ನು ಆರಿಸಬೇಕು? ಹೋಲಿಕೆ ಮತ್ತು ಮಿತಿಗಳು

ಆಪ್ಟಿಮೈಜರ್	ಮೂಲ ಕಲ್ಪನೆ	ಯಾವಾಗ ಒಳ್ಳೆಯದು	ಎಚ್ಚರ
Batch GD	ಇಡೀ ದತ್ತಾಂಶದ ಸರಾಸರಿ ಗ್ರೇಡಿಯಂಟ್	ಸಣ್ಣ ದತ್ತಾಂಶ, ಸ್ಥಿರತೆ ಬೇಕಾದಾಗ	ದೊಡ್ಡ ದತ್ತಾಂಶಕ್ಕೆ ನಿಧಾನ, ಮೆಮೊರಿ ಹೆಚ್ಚು
SGD	ಒಂದೊಂದು ಉದಾಹರಣೆ	ಹರಿವಿನ ದತ್ತಾಂಶ, ಬಹಳ ದೊಡ್ಡ ದತ್ತಾಂಶ	ಅಲುಗಾಟ; learning rate ಸೂಕ್ಷ್ಮ
Mini-batch GD	ಸಣ್ಣ ಗುಂಪು (32–128)	ಡೀಪ್ ಲರ್ನಿಂಗ್‌ನ ಪ್ರಮಾಣಿತ ಆಯ್ಕೆ	Batch size ಟ್ಯೂನ್ ಮಾಡಬೇಕು
Momentum / NAG	ಹಿಂದಿನ ವೇಗ ನೆನಪಿಡು (NAG: ಮುಂದೆ ನೋಡು)	ಅಲುಗಾಡುವ ಕಣಿವೆಗಳು, ಆಳ ನೆಟ್‌ವರ್ಕ್‌ಗಳು	ಗುರಿ ದಾಟಿ ಹೋಗಬಹುದು
AdaGrad	ಪ್ರತಿ ತೂಕಕ್ಕೆ ಬೇರೆ ದರ (ಒಟ್ಟು ವರ್ಗ ಮೊತ್ತ)	ವಿರಳ ದತ್ತಾಂಶ (ಪಠ್ಯ)	ಹೆಜ್ಜೆ ಕುಗ್ಗುತ್ತಲೇ ಹೋಗುತ್ತದೆ
RMSProp	ಮರೆಯುವ ಸರಾಸರಿ (AdaGrad ತಿದ್ದುಪಡಿ)	ಡೀಪ್ ನೆಟ್‌ವರ್ಕ್, RNN	—
Adadelta	learning rate ಬೇಕಿಲ್ಲ	ದರ ಹೊಂದಿಸಲು ಸಮಯವಿಲ್ಲದಾಗ	—
Adam	Momentum + RMSProp + bias ತಿದ್ದುಪಡಿ	**ಮೊದಲ ಪ್ರಯತ್ನಕ್ಕೆ ಉತ್ತಮ ಡೀಫಾಲ್ಟ್**	ಕೆಲವೊಮ್ಮೆ SGD+Momentum ಗಿಂತ ಸಾಮಾನ್ಯೀಕರಣ ಕಡಿಮೆ
Nadam	Adam + Nesterov	ಇನ್ನೂ ನಯವಾದ, ವೇಗದ ಕಲಿಕೆ	—

ಗ್ರೇಡಿಯಂಟ್ ವಿಧಾನದ ಮಿತಿಗಳು

- **ಆರಂಭಿಕ ಸ್ಥಾನದ ಮೇಲೆ ಅವಲಂಬನೆ:** ಆರಂಭಿಕ ತೂಕಗಳನ್ನು ಸಾಮಾನ್ಯವಾಗಿ ಯಾದೃಚ್ಛಿಕವಾಗಿ ಆರಿಸುತ್ತಾರೆ, ಹಾಗಾಗಿ ಪ್ರತಿ ಬಾರಿ ಬೇರೆ ಫಲಿತಾಂಶ ಬರಬಹುದು (ಬೇರೆ ಕಣಿವೆಗೆ ಇಳಿಯಬಹುದು).
- **ಹೆಜ್ಜೆಯ ಗಾತ್ರ (learning rate):** ತುಂಬ ಚಿಕ್ಕದಾದರೆ ನಿಧಾನ; ತುಂಬ ದೊಡ್ಡದಾದರೆ ಕನಿಷ್ಠವನ್ನು ದಾಟಿ ಹೋಗುತ್ತದೆ.

> **ಪ್ರಾಯೋಗಿಕ ಮಾರ್ಗದರ್ಶಿ**
>
> ೧) ಮರೆ ಪದರಗಳಿಗೆ **ReLU** (ಸಮಸ್ಯೆ ಬಂದರೆ Leaky ReLU/ELU). ೨) ಔಟ್‌ಪುಟ್ ಪದರ ಕಾರ್ಯಕ್ಕೆ ತಕ್ಕಂತೆ: ಸಂಖ್ಯೆಗೆ **Linear** + MSE/MAE; ಎರಡು ವರ್ಗಕ್ಕೆ **Sigmoid** + Binary Cross-Entropy; ಹಲವು ವರ್ಗಕ್ಕೆ **Softmax** + Categorical Cross-Entropy. ೩) ಆಪ್ಟಿಮೈಜರ್: **Adam ($\eta = 0.001$)** ದಿಂದ ಆರಂಭಿಸಿ, **mini-batch (32–128)** ಬಳಸಿ. ೪) ತರಬೇತಿ ಚೆನ್ನಾಗಿಲ್ಲದಿದ್ದರೆ learning rate ಬದಲಿಸಿ ನೋಡಿ.

೧೭. ಸಾರಾಂಶ, ಪದಕೋಶ ಮತ್ತು ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

ಪದಕೋಶ

ಇಂಗ್ಲಿಷ್	ಕನ್ನಡದಲ್ಲಿ ಸರಳ ಅರ್ಥ
Activation function	ನ್ಯೂರಾನ್‌ನ ಮೊತ್ತವನ್ನು ಔಟ್‌ಪುಟ್ ಆಗಿ ಬದಲಿಸುವ ಫಂಕ್ಷನ್; non-linearity ಕೊಡುತ್ತದೆ
Logit	Softmax/Sigmoid ಹಾಕುವ ಮೊದಲಿನ ಕಚ್ಚಾ ಅಂಕ
Vanishing gradient	ಹಿಂದಕ್ಕೆ ಹೋಗುತ್ತಾ ಗ್ರೇಡಿಯಂಟ್ ಕುಗ್ಗಿ ಆರಂಭದ ಪದರಗಳು ಕಲಿಯದಿರುವುದು
Dead neuron	ಯಾವಾಗಲೂ 0 ಕೊಡುವ (ReLU) ನ್ಯೂರಾನ್ — ಇನ್ನು ಕಲಿಯುವುದಿಲ್ಲ
Regression / Classification	ಸಂಖ್ಯೆ ಊಹೆ / ವರ್ಗ ಊಹೆ
Loss / Cost function	ಮಾದರಿ ಎಷ್ಟು ತಪ್ಪು ಎಂದು ಹೇಳುವ ಒಂದು ಸಂಖ್ಯೆ
MAE / MSE / RMSE	ಸರಾಸರಿ ಸಂಪೂರ್ಣ ತಪ್ಪು / ವರ್ಗ ತಪ್ಪು / ಅದರ ವರ್ಗಮೂಲ
MAPE	ಸರಾಸರಿ ಶೇಕಡಾ ತಪ್ಪು
Entropy	ಅನಿಶ್ಚಿತತೆ / ಮಿಶ್ರಣದ ಅಳತೆ
Cross-Entropy	ಊಹಿಸಿದ ಸಂಭವನೀಯತೆ ಮತ್ತು ನಿಜವಾದ ವರ್ಗದ ವ್ಯತ್ಯಾಸ ($-\ln p$)
Local minimum / Saddle point	ಗ್ರೇಡಿಯಂಟ್ 0 ಆಗಿರುವ ಆದರೆ ಅತ್ಯುತ್ತಮವಲ್ಲದ ಬಿಂದುಗಳು
Overfitting / Underfitting	ಉರು ಹೊಡೆಯುವುದು / ಸರಿಯಾಗಿ ಕಲಿಯದಿರುವುದು
Epoch	ಇಡೀ ತರಬೇತಿ ದತ್ತಾಂಶವನ್ನು ಒಮ್ಮೆ ನೋಡಿದ ಒಂದು ಸುತ್ತು
Batch / Mini-batch / SGD	ಇಡೀ ದತ್ತಾಂಶ / ಸಣ್ಣ ಗುಂಪು / ಒಂದೊಂದು ಉದಾಹರಣೆಯಿಂದ ತೂಕ ಬದಲಿಸುವುದು
Momentum	ಹಿಂದಿನ ಗ್ರೇಡಿಯಂಟ್‌ಗಳ ವೇಗವನ್ನು ಸಂಗ್ರಹಿಸಿ ಬಳಸುವುದು
Adaptive optimizer	ಪ್ರತಿ ತೂಕಕ್ಕೂ ತನ್ನದೇ ಆದ learning rate (AdaGrad, RMSProp, Adam…)

ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

1. ReLU(-3) ಮತ್ತು ReLU(2.5) ಎಷ್ಟು?
2. Sigmoid ಆಕ್ಟಿವೇಶನ್ ಇರುವ 3 ಪದರಗಳ ಮೂಲಕ ಹಿಂದಕ್ಕೆ ಹೋದಾಗ ಗರಿಷ್ಠ ಗ್ರೇಡಿಯಂಟ್ ಗುಣಕ ಎಷ್ಟು?
3. Softmax ಗೆ logits [1, 1, 1] ಕೊಟ್ಟರೆ ಔಟ್‌ಪುಟ್ ಎಷ್ಟು?
4. ನಾಳಿನ ತಾಪಮಾನ ಊಹಿಸಲು ಔಟ್‌ಪುಟ್ ಪದರದಲ್ಲಿ ಯಾವ ಆಕ್ಟಿವೇಶನ್? ಒಂದು ಚಿತ್ರದಲ್ಲಿ ಹಲವು ವಸ್ತುಗಳ ಟ್ಯಾಗ್ ಊಹಿಸಲು?

5. y = [10, 20], ŷ = [12, 18] ಆದರೆ MAE ಮತ್ತು MSE ಎಷ್ಟು?
6. y = 1 ಮತ್ತು ನೆಟ್‌ವರ್ಕ್ p = 0.5 ಆದರೆ binary cross-entropy loss ಎಷ್ಟು?
7. 600 ಉದಾಹರಣೆಗಳ ದತ್ತಾಂಶ, batch size 100 ಆದರೆ ಒಂದು epoch ನಲ್ಲಿ ತೂಕ ಎಷ್ಟು ಬಾರಿ ಬದಲಾಗುತ್ತದೆ? SGD ಯಲ್ಲಿ? Batch GD ಯಲ್ಲಿ?
8. ಯಾವ ಆಪ್ಟಿಮೈಜರ್ ಅನ್ನು "ಮೊದಲ ಪ್ರಯತ್ನಕ್ಕೆ ಉತ್ತಮ ಡೀಫಾಲ್ಟ್" ಎನ್ನುತ್ತಾರೆ, ಮತ್ತು ಅದರ ಸಾಮಾನ್ಯ learning rate ಎಷ್ಟು?

ಉತ್ತರಗಳು: (೧) 0 ಮತ್ತು 2.5. (೨) 0.25^3 = **0.0156**. (೩) ಮೂರೂ ಸಮ: **[1/3, 1/3, 1/3] ≈ [0.333, 0.333, 0.333]**. (೪) ತಾಪಮಾನಕ್ಕೆ **Linear**; ಬಹು ಟ್ಯಾಗ್‌ಗೆ **ಪ್ರತಿ ಲೇಬಲ್‌ಗೆ Sigmoid**. (೫) ತಪ್ಪುಗಳು −2, +2: MAE = (2+2)/2 = **2**; MSE = (4+4)/2 = **4**. (೬) $-\ln(0.5)$ = **0.693**. (೭) Mini-batch: 600/100 = **6** ಬಾರಿ; SGD: **600** ಬಾರಿ; Batch GD: **1** ಬಾರಿ. (೮) **Adam**, learning rate **0.001**.